

AuditPlatform

Rapport d'analyse — plateforme-audit-gitlab

Généré le 16/04/2026 à 11:59

■ Informations du projet

Nom du projet	plateforme-audit-gitlab
URL du projet	yosrchiha01/plateforme-audit-gitlab
Branche analysée	main
Date de l'analyse	16/04/2026 11:29
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	47	47/100
Sécurité	47	47/100
Performance	20	20/100

■ ■ Vulnérabilités (40)

A02 — CRYPTOGRAPHIC FAILURES
Sévérité : CRITIQUE
Fichier : backend/app/config/mail.py — ligne 9
Correction : Utiliser un mot de passe sécurisé pour l'application Gmail
A02 — CRYPTOGRAPHIC FAILURES
Sévérité : CRITIQUE
Fichier : backend/app/core/security.py — ligne 8

Correction : Utiliser un algorithme de hachage sécurisé pour les mots de passe

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : CRITIQUE

Fichier : backend/app/core/security.py — ligne 25

Correction : Utiliser un algorithme de chiffrement sécurisé pour les données sensibles

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : CRITIQUE

Fichier : backend/send_test_mail.py — ligne 7

Correction : Utiliser un mot de passe sécurisé pour l'application Gmail

A05 — SECURITY MISCONFIGURATION

Sévérité : HAUTE

Fichier : backend/app/config/database.py — ligne 16

Correction : Désactiver le mode debug en production

A05 — SECURITY MISCONFIGURATION

Sévérité : HAUTE

Fichier : backend/app/config/settings.py — ligne 33

Correction : Utiliser un fichier de configuration sécurisé pour les variables d'environnement

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : backend/app/routes/auth.py — ligne 39

Correction : Utiliser un mécanisme d'authentification sécurisé pour les utilisateurs

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : backend/app/routes/auth.py — ligne 85

Correction : Utiliser un mécanisme de protection contre le brute force pour les utilisateurs

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : backend/app/routes/auth.py — ligne 136

Correction : Utiliser un mécanisme de réinitialisation de mot de passe sécurisé pour les utilisateurs

A03 — INJECTION

Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 31

Correction : Utiliser un mécanisme de validation des entrées utilisateur pour éviter les injections

A03 — INJECTION

Sévérité : HAUTE

Fichier : backend/app/routes/depots.py — ligne 103

Correction : Utiliser un mécanisme de validation des entrées utilisateur pour éviter les injections

A10 — SSRF (SERVER-SIDE REQUEST FORGERY)

Sévérité : HAUTE

Fichier : backend/app/routes/gitlab.py — ligne 6

Correction : Utiliser un mécanisme de validation des URLs pour éviter les attaques SSRF

A03 — INJECTION

Sévérité : HAUTE

Fichier : frontend/app/add-repository/page.tsx — ligne 21

Correction : Utiliser un mécanisme de validation des entrées utilisateur pour éviter les injections

A03 — INJECTION

Sévérité : HAUTE

Fichier : frontend/app/components/Input.tsx — ligne 16

Correction : Utiliser un mécanisme de validation des entrées utilisateur pour éviter les injections

A03 — INJECTION

Sévérité : HAUTE

Fichier : frontend/app/dashboard/page.tsx — ligne 45

Correction : Utiliser un mécanisme de validation des entrées utilisateur pour éviter les injections

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Utiliser HTTPS pour les requêtes sensibles, comme la réinitialisation de mot de passe.

A05 — SECURITY MISCONFIGURATION

Sévérité : HAUTE

Fichier : frontend/next.config.ts — ligne 1

Correction : Configurer correctement le fichier next.config.ts pour inclure les options de sécurité nécessaires, comme l'utilisation de HTTPS.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 16

Correction : Diviser la fonction handleSubmit en plusieurs fonctions plus petites pour améliorer la lisibilité et la maintenabilité.

Maintenabilité

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Extraire l'URL de l'API en une constante pour faciliter les modifications futures.

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 26

Correction : Implémenter une politique de mot de passe plus robuste, comme l'utilisation de caractères spéciaux, de chiffres et de lettres majuscules.

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 21

Correction : Implémenter une vérification de mot de passe plus robuste, comme l'utilisation d'une bibliothèque de vérification de mot de passe.

Timeout absent

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Ajouter un timeout pour les appels HTTP, par exemple avec `fetch` et `AbortController`, pour éviter les blocages indéfinis. Gain estimé : 10% de réduction des temps de chargement

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 6

Correction : Ajouter une docstring pour décrire la fonction ResetPassword

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 16

Correction : Ajouter une docstring pour décrire la fonction handleSubmit

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 31

Correction : Ajouter un commentaire pour décrire le bloc de code try-catch

Gestion erreurs

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 55

Correction : Utiliser des exceptions spécifiques au lieu de catch génériques pour une meilleure gestion des erreurs

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 21

Correction : Utiliser des noms de variables plus descriptifs pour les conditions, par exemple 'passwordMismatch' au lieu de 'password !== confirmPassword'.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 115

Correction : Utiliser des noms de propriétés plus descriptifs pour les styles, par exemple 'containerStyles' au lieu de 'container'.

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 0

Correction : Ajouter des commentaires pour expliquer le but et les options de configuration de next.config.ts.

A09 — LOGGING AND MONITORING FAILURES

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 55

Correction : Implémenter une journalisation plus robuste pour les erreurs, comme l'utilisation d'une bibliothèque de journalisation.

Cache absent

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Implémenter un mécanisme de cache pour les données de réinitialisation de mot de passe, si les données sont rarement modifiées. Gain estimé : 5% de réduction des temps de chargement

Boucle $O(n^2)$

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 16

Correction : Optimiser la fonction `handleSubmit` pour éviter les boucles imbriquées inutiles.
Gain estimé : 2% de réduction des temps de chargement

Mémoire

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 3

Correction : Vérifier les options de configuration de Next.js pour optimiser la gestion de la mémoire. Gain estimé : 1% de réduction des temps de chargement

Ressource

Sévérité : FAIBLE

Fichier : frontend/next.config.ts — ligne 5

Correction : Vérifier les options de configuration de Next.js pour optimiser la gestion des ressources. Gain estimé : 1% de réduction des temps de chargement

Commentaire absent

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 9

Correction : Ajouter un commentaire pour décrire la variable email

Commentaire absent

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 10

Correction : Ajouter un commentaire pour décrire la variable otp

Commentaire absent

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 11

Correction : Ajouter un commentaire pour décrire la variable password

Commentaire absent

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 12

Correction : Ajouter un commentaire pour décrire la variable confirmPassword

SOLID-D

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 34

Correction : Utiliser une injection de dépendance pour la gestion des requêtes HTTP au lieu d'une implémentation concrète

Testabilité

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 115

Correction : Définir les styles en dehors de la fonction pour améliorer la testabilité

■ Recommandations

1. Utiliser un mécanisme de validation des entrées utilisateur pour éviter les injections

Il est important de valider les entrées utilisateur pour éviter les injections SQL, les injections de commandes OS et les attaques XSS. Cela peut être fait en utilisant des bibliothèques de validation telles que Sanitize ou en écrivant des fonctions de validation personnalisées.

2. Utiliser un mécanisme de protection contre le brute force pour les utilisateurs

Il est important de protéger les utilisateurs contre les attaques de brute force en utilisant des mécanismes de protection telles que les compteurs de tentative de connexion ou les mécanismes de blocage temporaire.

3. Utiliser un mécanisme de réinitialisation de mot de passe sécurisé pour les utilisateurs

Il est important de réinitialiser les mots de passe des utilisateurs de manière sécurisée en utilisant des mécanismes de réinitialisation de mot de passe tels que les codes OTP ou les questions de sécurité.

4. Utiliser un mécanisme de validation des URLs pour éviter les attaques SSRF

Il est important de valider les URLs pour éviter les attaques SSRF en utilisant des bibliothèques de validation telles que Sanitize ou en écrivant des fonctions de validation personnalisées.

5. Améliorer la structure des fonctions

Diviser les fonctions longues en plusieurs fonctions plus petites pour améliorer la lisibilité et la maintenabilité. Par exemple, la fonction `handleSubmit` pourrait être divisée en plusieurs fonctions pour gérer les différentes étapes de la réinitialisation du mot de passe.

6. Utiliser des noms de variables descriptifs

Utiliser des noms de variables plus descriptifs pour améliorer la lisibilité du code. Par exemple, au lieu d'utiliser `'x'` ou `'tmp'`, utiliser des noms de variables qui reflètent leur contenu ou leur but.

7. Utilisation de HTTPS

Il est recommandé d'utiliser HTTPS pour les requêtes sensibles, comme la réinitialisation de mot de passe, pour protéger les données des utilisateurs.

8. Implémentation d'une politique de mot de passe robuste

Il est recommandé d'implémenter une politique de mot de passe plus robuste, comme l'utilisation de caractères spéciaux, de chiffres et de lettres majuscules, pour protéger les comptes des utilisateurs.

9. Utilisation d'une bibliothèque de vérification de mot de passe

Il est recommandé d'utiliser une bibliothèque de vérification de mot de passe pour implémenter une vérification de mot de passe plus robuste.

10. Implémentation d'une journalisation robuste

Il est recommandé d'implémenter une journalisation plus robuste pour les erreurs, comme l'utilisation d'une bibliothèque de journalisation, pour faciliter la détection et la correction des problèmes de sécurité.

11. Optimiser les appels HTTP

Utiliser des bibliothèques comme ``axios`` ou ``fetch`` avec des options de timeout pour éviter les blocages indéfinis. Exemple : ``fetch(url, { timeout: 5000 })``

12. Implémenter un mécanisme de cache

Utiliser des bibliothèques comme ``react-query`` ou ``redux`` pour implémenter un mécanisme de cache pour les données rarement modifiées. Exemple : ``useQuery('donnees', fetchDonnees)``

13. Optimiser les fonctions

Utiliser des outils comme ``eslint`` ou ``prettier`` pour détecter les boucles imbriquées inutiles et les optimiser. Exemple : ``for (let i = 0; i < array.length; i++) { ... }``

14. Ajouter des docstrings

Ajouter des docstrings pour décrire les fonctions et les variables pour améliorer la lisibilité du code

15. Ajouter des commentaires

Ajouter des commentaires pour décrire les blocs de code complexes pour améliorer la compréhension du code

16. Utiliser des noms de variables clairs

Utiliser des noms de variables clairs et descriptifs pour améliorer la lisibilité du code

17. Refactorisation de la gestion des erreurs

Utiliser des exceptions spécifiques pour chaque type d'erreur, par exemple, une exception pour les erreurs de validation des données et une autre pour les erreurs de réseau. Cela permettra une gestion plus fine et plus efficace des erreurs.

18. Amélioration de la testabilité

Définir les styles en dehors de la fonction pour améliorer la testabilité. Cela permettra de tester les composants sans avoir à gérer les styles. De plus, utiliser des mocks pour les requêtes HTTP pour améliorer la testabilité de la fonction.

19. Utilisation de l'injection de dépendance

Utiliser une injection de dépendance pour la gestion des requêtes HTTP au lieu d'une implémentation concrète. Cela permettra de changer facilement l'implémentation des requêtes HTTP sans avoir à modifier le code de la fonction.